

Esercizio B1 – Punti ≥ 8

L'esercizio è praticamente identico all'esercizio "Prima elementare" (Compito 01/02/2012), quello in cui si chiedeva di stampare n caratteri R e m caratteri G (per indicare palline gialle e palline rosse). Il punto è che per arrivare in posizione (n, n) , bisognerà produrre n passi "D" e n passi R .

Si utilizza la tecnica del backtrack; se non esistono più lettere da collocare, abbiamo terminato la nostra stringa e la stampiamo. Altrimenti, se esiste la possibilità di aggiungere una lettera "D" si prova ad aggiungerla e si chiama la funzione ricorsiva con una lettera D in meno ($n_d - 1$); in modo simile si opera sulla lettera "R". Il numero di stringhe da stampare (ognuna lunga $2n$) è pari a $\frac{(2n)!}{n!n!}$. La complessità è quindi $O\left(\frac{n \cdot (2n)!}{n!n!}\right)$. È comunque corretto dire che la complessità è $O(n2^{2n})$, perché vengono fatti $2n$ scelte, ognuna delle quali ha due possibilità. Non è corretto dire che la complessità è $O(n2^n)$, perché non si può semplificare dicendo che $O(2^n) = O(2^{2n})$, in quanto $O(2^{2n}) = O(4^n)$.

```
printPaths(int n)
```

```
    S = new ITEM[1...2n]
    printRec(S, 1, n, n)
```

```
printRec(char[] S, int i, int n_d, int n_r)
```

```
    if n_d == 0 and n_r == 0 then
        | print S
    else
        | if n_d > 0 then
            | | S[i] = "D"
            | | printRec(S, i + 1, n_d - 1, n_r)
        | if n_r > 0 then
            | | S[i] = "R"
            | | printRec(S, i + 1, n_d, n_r - 1)
```

Esercizio B2 – Punti ≥ 10

Questo esercizio è praticamente il duale dell'Esercizio B2 del 08/02/2021 (Zaino leggero). Invece di individuare il sottoinsieme che la cui somma è C e ha dimensione più piccola in assoluto, si deve individuare il sottoinsieme di elementi da rimuovere di dimensione più grande per cui la somma restante è la più grande possibile. È quindi possibile riutilizzare la soluzione dell'esercizio Zaino Leggero, ottenendo la dimensione dell'insieme più piccolo e quindi sottraendo tale valore dalla dimensione n .

```
int removeSum(int[] A, int n, int C)
```

```
    return n - smallSum(A, n, C)
```

Si noti che l'algoritmo proposto nel compito del 08/02/2021 restituisce $+\infty$ se non è possibile ottenere C ; in questo caso, restituirà $-\infty$ ottenuto dalla sottrazione di $+\infty$ da n . La complessità è quella dell'esercizio del 08/02/2021, ovvero $O(nC)$

Volendo riscrivere invece la soluzione da zero, ripropongo solo la formula:

$$DP[i][c] = \begin{cases} i & c = 0 \\ -\infty & c < 0 \\ -\infty & c > 0 \wedge i = 0 \\ \max(DP[i-1][c] + 1, DP[i-1][c - W[i]]) & c > 0 \wedge i > 0 \end{cases}$$

Notate che se $c = 0$, significa che si è ottenuto il valore C senza utilizzare gli elementi negli indici compresi fra 1 e i ; per questo motivo, è come se venissero rimossi e quindi $DP[i][c] = i$ in quel caso.

Esercizio B3 – Punti ≥ 12

Questo esercizio è simile all'Esercizio 1.5 del file di esercizi sulla programmazione dinamica, con la differenza che non consideriamo la lunghezza delle sequenza crescente più lunga, ma la somma dei suoi valori. Nella definizione ricorsiva, basta sostituire tutte le occorrenze di 1 con $A[i]$. La versione presentata qui sotto semplifica ulteriori aspetti.

Denotiamo con $DP[i]$ il valore massimo fra tutte le sottosequenza crescenti che terminano nella posizione i -esima. È possibile calcolare $DP[i]$ in maniera ricorsiva. Si consideri l'indice i e si considerino gli elementi minori di $A[i]$ nel sottovettore $A[1 \dots i - 1]$; $A[i]$ può essere utilizzato per estendere la sottosequenza di valore massimo che termina in uno di questi elementi. Se non esistono elementi minori, allora dobbiamo "ricominciare da capo", ovvero considerare la sequenza composta dal singolo valore $A[i]$.

Un modo per esprimerlo è il seguente:

$$DP[i] = \begin{cases} A[i] & \forall j, 1 \leq j < i : A[i] < A[j] \\ \max\{DP[j] : 1 \leq j \leq i - 1 \wedge A[j] < A[i]\} + A[i] & \text{altrimenti} \end{cases}$$

Per risolvere il problema, utilizziamo programmazione dinamica. La funzione calcola tutti i valori $DP[i]$ con $i = 1 \dots n$, in ordine. Al termine, si restituisce il massimo fra i valori memorizzati in DP . Il costo della procedura è $O(n^2)$.

```

int maxSumIncreasing(int[] A, int n)
{
    int[] DP = new int[1 .. n]
    for  $i = 1$  to  $n$  do
    {
        DP[i] = A[i]           % Se non troviamo valori minori, consideriamo solo questo elemento
        for  $j = 1$  to  $i - 1$  do
        {
            if  $A[j] < A[i]$  and  $DP[j] + A[i] > DP[i]$  then
            {
                DP[i] = DP[j] + A[i]
            }
        }
    }
    return max(DP)
}

```

In più, è stato anche proposto come esercizio B3 del 18/07/19.